

Article

A Hardware Non-Invasive Mapping Method for Condition Bits in Binary Translation

Chunqiang Li ^{1,*}, Zhiwei Liu ^{2,*} , Yunhai Shang ², Lenian He ¹ and Xiaolang Yan ¹¹ Institute of VLSI Design, Zhejiang University, Hangzhou 310000, China² Alibaba Group, Hangzhou 310000, China

* Correspondence: chunqiangli@163.com (C.L.); baxiantai@gmail.com (Z.L.)

Abstract: Binary translation, as an important bridge for application compatibility between different instruction set architectures (ISAs), has attracted much attention in the industry. However, due to hardware resource limitations of the target ISA, the translation efficiency and the practicability are poor. Recently, Apple has made it possible to run x86 programs on ARM through a translation technology called Rosetta based on software-hardware collaboration. In this paper, we proposed a hardware non-invasive mapping method for condition bits (HNIMCB) in binary translation, which innovatively implements the setting and referencing operations of the condition bits without changing the original instruction encoding and function of the target processor. This method is applicable for binary translation from source architectures with condition bit operations to target architectures without condition bit operations. It eliminates the difference of conditional bit resources between the source and target ISAs, reduces the computational instructions and memory access operations after translation from the source to the target ISA, and dramatically improves the translation efficiency. We conducted this experiment on a functional simulation level using the QEMU binary translator from ARM to RISC-V. A series of benchmark tests revealed that the total number of instructions decreased by 41%, while the number of memory access instructions decreased by 37% after the translation applying with the HNIMCB.

Keywords: binary translation; condition bit; non-invasive mapping; instruction set architecture; RISC-V; ARM; QEMU



Citation: Li, C.; Liu, Z.; Shang, Y.; He, L.; Yan, X. A Hardware Non-Invasive Mapping Method for Condition Bits in Binary Translation. *Electronics* **2023**, *12*, 3014. <https://doi.org/10.3390/electronics12143014>

Academic Editor: Akash Kumar

Received: 29 May 2023

Revised: 4 July 2023

Accepted: 7 July 2023

Published: 9 July 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The development of instruction set architecture (ISA) has never stopped, typically with x86 monopolizing the desktop and server markets, and ARM dominating the mobile computing market, respectively. The design of these ISAs usually differ, and respective applications are often incompatible with each other, thereby presenting a large obstacle to the development of ISA. To solve this problem, there have been many research papers related to virtualization [1–3] and the binary translation of the ISAs.

In recent years, RISC-V has developed rapidly with widespread attention from ecosystem developers for its open ecosystem and scalable, customizable ISA features. In the years after its creation in 2010, RISC-V has been mainly used in specialized chips, such as power management and the RF protocol. With the gradual improvement in the basic RV32I and RV32E standard instruction sets, more and more SoC chips of the IoT and MCU are reportedly using RISC-V. In 2022, the shipment of RISC-V processors exceeded 10 billion, and the main work of the RISC-V International Foundation is moving from technology improvement to other key related areas, such as cloud computing, edge computing, and automotive. The RISC-V software ecosystem has also evolved rapidly from the bare metal programs for specialized chips to RTOS for MCUs. Especially in recent years, strong UI interactions and high-performance computing OS on AP chips, such as Android, Ubuntu, Fedora, Anolis, and Kirin already support RISC-V [4,5]. However, in the Android and Linux

systems, their applications are still dominated by x86 and ARM, which greatly restricts the further development of the RISC-V ecosystem.

To further accelerate the integration of RISC-V and high-performance application ecosystems, a binary translator based on RISC-V is deemed as a viable path for rapid adaptation to the existing ecosystems. Binary translation techniques have been widely implemented in computer architecture research and commercial applications. Early implementations include FX!32 [6], combined with dynamic translation and static translation, IA-32 EL [7], realized by Intel for IA32 applications running on an IA64 system, and UQBT [8], developed by the University of Queensland supporting multi-source and multi-target ISAs. The company Transmeta directly implemented the x86-like condition bit registers on its own VLIW processor Crusoe so that the binary translator CodeMorphing [9] can directly map the condition bits instructions from x86 to Crusoe, and can reduce the computational complexity caused by the setting and referencing of the condition bits. Currently, there are many open-source binary translation implementations, the most typical of which are QEMU [10], Box64 [11], FEX-EMU [12], and Instrew [13], etc. In recent years, there are several binary translators, such as Apple's Rosetta and Intel's Houdini, used commercially due to their excellent translation efficiency.

Binary translation between these ISAs can be categorized into universal compilation technology. It includes the front-end, middle-end, and back-end, and implements a variety of optimization algorithms at each end. Its architecture covers an interpretive execution mode, a static translation mode, and a dynamic just-in-time translation mode, etc. In particular implementation, several modes are often combined to improve the execution performance of the translated target code. Due to the loss of information like indirect jumps information, variable life cycle, and register allocation [14–18] in the source binary code, the optimization effect of pure software binary translation is quite limited. The mainstream optimization methodology includes register allocation and condition bit operation in translation, as well as the runtime native library call techniques. Among them, there has been much research performing regarding the conditional bit operation optimization technology of pure software, but few have been conducted on efficiency improvement. In particular, the target ISA, such as Alpha, MIPS, and RISC-V lacks condition bit registers, as well as the resources for the setting and referencing of the condition bits. Thus, additional computation instruction and memory access overhead are both required to achieve the mapping relationship between the source and target instructions when translating the condition bits instructions of source ISAs with multiple condition bits, such as x86 and ARM.

This paper takes the translation of ARM to RISC-V as an example. The ARM ISA contains N/Z/C/V condition bits, which will be set or referenced by various arithmetic instructions and comparison instructions, and even referenced by conditional codes in instruction encoding. Since RISC-V ISA design usually takes simplicity [4,5] into account, there is no design of the condition bit register, no setting, and no referencing of the condition bits, and usually comparisons and branch functions are completed directly in one instruction. As shown in Figure 1, this difference makes the translation from an ARM's setting and referencing instructions of the condition bits to RISC-V instructions requiring a lot of computation instructions and memory access operations.

As shown in Figure 1, the translated code fragment was obtained from the QEMU emulator. For the two-ARM condition bit instructions shown, it required 16 RISC-V instructions to simulate. This highlights the significant cost of translating condition bit instructions. Additionally, we can see that for the same source code, using a native RISC-V compiler requires only one RISC-V instruction, which is much more efficient than using binary translation. This indicates that binary translation techniques, like QEMU, are not competitive with native compilers, and we need to use a more efficient method for condition bit instruction translation.

In order to analyze the impact of translation optimization for condition bit instructions on binary translation efficiency, we have recorded the number and proportions of condition

bit instructions during the dynamic execution of the SPEC 2006 benchmark on ARM A64, as shown in Table 1.

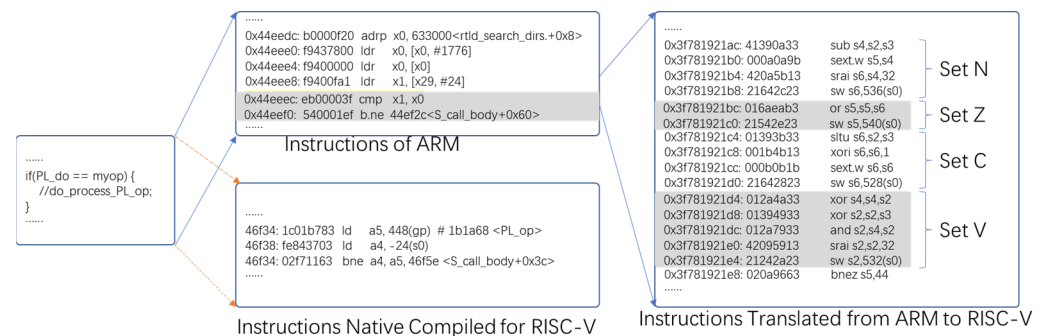


Figure 1. Translation overhead for condition bit instructions.

Table 1. ARM SPEC 2006 condition bit instructions ratio.

Application	Total Insn	Total CC	CC SET	CC REF	% (CC)	% (CC_SET)	% (CC_REF)
400.perlbench	16,400,349,170	2,098,384,522	1,056,190,851	1,042,193,671	12.79	6.44	6.35
401.bzip2	16,384,928,864	2,509,858,452	1,226,489,918	1,283,368,534	15.32	7.49	7.83
403.gcc	6,934,946,632	1,049,091,684	526,159,467	522,932,217	15.13	7.59	7.54
429.mcf	5,241,171,907	1,278,287,501	641,790,993	636,496,508	24.39	12.25	12.14
445.gobmk	26,588,139,331	3,226,104,557	1,594,047,318	1,632,057,239	12.13	6	6.14
456.hmmer	24,694,491,366	4,060,210,434	1,950,169,823	2,110,040,611	16.44	7.9	8.54
458.sjeng	8,234,136,005	1,351,115,281	647,076,640	704,038,641	16.41	7.86	8.55
462.libquantum	11,197,746,346	3,897,060,080	1,947,416,491	1,949,643,589	34.8	17.39	17.41
464.h264ref	34,656,333,006	2,389,042,712	1,205,872,292	1,183,170,420	6.89	3.48	3.41
471.omnetpp	2,778,049,010	196,724,662	93,354,907	103,369,755	7.08	3.36	3.72
473.astar	33,500,059,582	6,398,981,406	3,277,740,756	3,121,240,650	19.1	9.78	9.32
483.xalancbmk	546,593,329	121,517,269	60,711,144	60,806,125	22.23	11.11	11.12
433.milc	3,402,485,964	117,724,688	67,329,738	50,394,950	3.46	1.98	1.48
444.namd	83,297,604,562	4,670,779,842	1,860,331,032	2,810,448,810	5.61	2.23	3.37
447.dealII	14,792,890,116	1,339,095,870	666,545,713	672,550,157	9.05	4.51	4.55
450.soplex	11,261,774,688	1,925,777,438	846,201,165	1,079,576,273	17.1	7.51	9.59
453.povray	4,338,040,149	213,084,315	76,578,923	136,505,392	4.91	1.77	3.15
470.lbm	1,334,745,166	44,956,902	23,805,633	21,151,269	3.37	1.78	1.58
482.sphinx3	11,503,902,542	1,217,056,110	612,207,866	604,848,244	10.58	5.32	5.26

According to Table 1, the proportion of condition bit instructions in ARM SPEC 2006 programs peaks at 34.8%, with an average proportion of 13%. As shown in Figure 1, due to the differences between the ARM and the RISC-V ISAs, a series of RISC-V arithmetic and memory access instructions are therefore required to translate a single ARM condition bit instruction, which severely affects their translation performance. All of these indicate the significant importance of optimizing the translation of the condition bit instructions for the ARM to RISC-V binary translation.

The difference of condition bits resources between the source and the target ISAs leads to a large number of computational instructions and memory access operations in the translation of the setting and the referencing of the condition bits. To solve the problem, this paper proposed a hardware non-invasive mapping method for condition bits (HNIMCB) in binary translation. By adding a new execution mode to the target ISA, this method expands the condition bit register which is mapped one-to-one with the condition bit of the source ISA and realizes the setting and referencing of the condition bits on the original target ISA without destroying the standard instruction set and programming model. Thus, this method achieves the efficient translation of the condition bit instructions from the source ISA to the target ISA, effectively reduces the translation complexity, and at the same time, reduces the count of computation instructions and memory access operations after mapping.

In Section 1, we analyze the problem of the translation efficiency of the setting and referencing of these condition bits and put forward the corresponding solutions. In Section 2, we analyze the current mainstream binary translator, especially in the translation and optimization of the setting and referencing of the condition bits. Section 3 introduces the general framework and the implementation details of applying the HNIMCB in binary translation. The experimental data on a functional simulation level were presented and analyzed in Section 4. Section 5 evaluates the impact of the microarchitecture and the cost of extending the technique to X86 translation. In Section 6, the overall effects and significance of the proposed method are summarized.

2. Related Work

QEMU [10] is a typical, open-source binary translator that supports dynamic JIT translation technology. It supports the user mode and the system mode, and improves performance through basic block translation, translation cache, and TB call chains. QEMU achieves simple optimization in terms of the setting and referencing of condition bits and does some combination optimization of the setting and storage of the condition bits within a basic block. However, it still cannot avoid the situation in that the source ISA has abundant condition bits, while the target ISA does not contain the condition bit resources, leading to a low translation efficiency. For example, when the ARM binary code is translated into RISC-V, a large number of computation instructions and memory access operations are generated as a result due to the setting and referencing of the condition bits.

Box86 [11] enables the running of x86 Linux applications on non-x86 Linux systems, including ARM. Its performance is enhanced by two key features. Firstly, the native library twist allows the program to utilize native versions of the “system” libraries, like libc, libm, SDL, and OpenGL, on the target platform. Secondly, the JIT engine (Dynatec) provides a speed boost of five-to-ten times faster than only using an interpreter. In terms of the setting and referencing of the condition bits, Dynatec checks these operations for each instruction in one basic block, and then each instruction knows if and which condition bits must be set after its execution. This condition bit propagation technique of Dynatec effectively avoids unnecessary calculations of the condition bits.

FEX-EMU [12] is a specialized emulator designed for AArch64 that enables the execution of the x86 and x86-64 games. It has several features which contribute to achieving performance levels that are only 25% to 50% slower than the native code, including the native libraries call technology, offline compilation, and tooling for performance analysis, etc. The emulator also implements several IR optimizations passes, one of which is the dead condition bit elimination. This pass eliminates the redundant calculations of the condition bits that are overwritten without being used. It breaks out the condition bits to independent memory locations to reuse the store elimination optimization in three steps: computing which condition bits are read and written per block, determining which condition bits are stored but will be overwritten by the next block(s), and finally removing the dead stores.

Instrew [13] is an LLVM-based dynamic binary translator that utilizes LLVM’s mature and efficient optimizations [19]. It converts the source binary code to LLVM IR through a process called lifting, which results in IR functions rather than basic blocks or superblocks. This makes non-local optimizations possible and reduces the number of jumps between the code blocks. During the lifting stage, it divides the condition bit register into seven stored flags (sign, zero, carry, overflow, parity, adjust, and direction) as these flags are typically written and evaluated independently of each other, and are rarely used in the format outlined by the x86 architecture. In addition, the flag evaluation is optimized so that subsequent condition queries, like those in jumps, can easily be folded into a single LLVM ‘icmp’ instruction during optimization. In the translation stage, within an LLVM function, the dead code elimination pass will be used to remove the compilation of the condition bits that are not used by the succeeding code. Another optimization in the translation stage for the condition bits is that all the condition bits are discarded when a call or ret instruction is

encountered, since no compiler or calling conversion necessitates condition bit preservation over the function boundaries.

Rosetta is an early version of Apple's commercial binary translator that allows applications to migrate from the PPC to the x86 architecture. In 2020, Apple released the ARM-based M1 chip, and at the same time, delivered Rosetta2 to run x86 applications on the ARM ISA. According to the data provided by Apple on Geekbench's official website [20], the single core scores of the M1 chips running x86 binary programs with Rosetta2 are 1313, compared with the single core scores of 1687 directly running on the M1 chip, respectively. Rosetta2, running x86 code, achieves a 78% straight-up performance. It is worth learning that the excellent performance of Rosetta2 is not only due to native library mapping optimization, RAS indirect jump elimination, and other methods, but also due to the combination of the software and hardware translation architecture and the AOT + JIT architecture. Apple designs the hardware acceleration of instructions for binary translation in the M1 chip to achieve a direct one-to-one mapping of the instructions. In addition, although ARM is a RISC ISA, it has similar condition bits and operations to x86. Therefore, Rosetta2 does not need to do much work on the translation of the condition bit operations and can directly carry out register mapping on x86 condition bits, reducing the complexity of calculation and memory access caused by the setting and referencing of the condition bits.

Ma Xiangning et al. [21] proposed a method with combining real-time computation and delayed computation, and with combining data flow analysis and delayed computation to optimize the computation and memory access cost of the target code from the dynamic translation of the setting and referencing of the condition bits. This method effectively reduces the target code caused by redundant condition bit changes. However, the translation cost caused by data flow analysis during the dynamic translation should not be ignored. Meanwhile, complex computation instructions and memory access operations are still needed for necessary condition bit setting and referencing on the target ISA in this method. Tang Feng et al. [22] proposed a linear analysis method of the condition bits for successor basic blocks to obtain the relationship between the basic blocks for the setting and the referencing of the condition bits. This method further reduced the redundant calculation of the condition bits. However, the analysis of successor base blocks incurs additional translation overhead, and this method does not effectively reduce the overhead caused by the computation and memory access for the setting and referencing of the condition bits in the dynamic run time. Wang Wenwen et al. [23] proposed a method of condition bit pattern search and translation. This method optimized the setting and referencing instruction sequences of the necessary condition bits on the target ISA to a certain extent, but further increased the overhead in the dynamic translation and failed to deal with the complex condition bit patterns and their distributions. Wang Ronghua et al. [24] proposed an efficient mapping method termed the compare and condition branch fast mapping algorithm. This algorithm focuses on the "compare and condition branch" instruction pairs which occupy a large proportion of the program, and it realizes the efficient mapping of the "compare and condition branch" instruction pairs using the inherent conditional dependencies of the target ISA. This algorithm improves the efficiency of the dynamic translation and execution by avoiding the complex and uniform traditional processes for these special instruction pairs. However, in the data flow analysis of the dynamic translation, this method needs to store multiple information of each condition bit setting instruction, which brings extra memory access overhead.

The current binary translation for condition bit operations mostly adopts the method of delayed computation to reduce the condition bit computation, the pattern matching to reduce the number of instructions for the setting and referencing of the condition bit, and data flow analysis to implement optimization methods, such as deleting the redundant calculations of the condition bit. However, these methods bring new problems while optimizing the translation efficiency. The delayed computation method needs to save the instruction code, operands, and other information, which increases the memory

access consumption during dynamic execution. Pattern matching and data flow analysis algorithms increase the complexity of translation. Moreover, in the necessary condition bit operation of the source instructions, a large number of computation instructions and memory access are still needed to achieve the setting and referencing of the condition bits. In general, the current binary translation has mainly been based on the existing hardware implementation, and mostly uses pure software translation frameworks and optimization techniques. This makes it difficult to achieve the commercial indicators for the translation efficiency when there are significant differences between the source and the target ISAs. In recent years, in order to solve the limitations of pure software translation optimization, there have gradually been some research performed on improving the efficiency of binary translation through software-hardware collaboration technology, such as Apple's Rosetta, which makes it possible to commercialize translation between the x86 and ARM. For a RISC-V open ISA, this kind of software-hardware collaboration optimization technology is precisely in the innovative direction of binary translation.

3. Design

3.1. Implementation Framework

In this paper, we proposed a hardware non-invasive mapping method for condition bits in binary translation. The purpose of using this method was to provide hardware resources to achieve an efficient mapping between the source and the target ISA for binary translation without modifying the target ISA and the programming model. In view of the problems summarized in Section 2, we need to effectively reduce the complexity of translation and reduce the condition bit calculation instructions and memory access operations through the one-to-one mapping of the condition bit registers, setting, and referencing between the source and the target architecture. In order to achieve the above purpose, this paper added a dynamic execution mode to the target ISA, and cleverly realizes the switching mechanism between the translation mode and the dynamic execution mode on the existing ISA. Figure 2 illustrates the design framework of the hardware non-invasive mapping method for condition bits in binary translation.

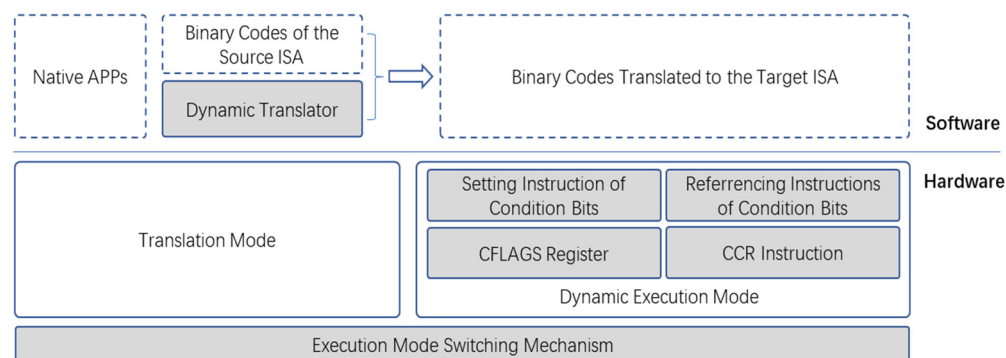


Figure 2. The HNIMCB in binary translation.

The mode switching mechanism is responsible for switching between the translation mode and the dynamic execution mode. In the translation mode, the running resources of the target processor are fully compatible with the original target ISA, and the ordinary native applications and the dynamic translator are run in this mode. In the dynamic execution mode, the target processor is extended with the condition bit register CFLAGS, and innovatively introduces the identifier instruction for the condition bit (CCR instruction). Thus, the successor arithmetic instructions following the CCR instruction can carry out the function expansion based on the semantics of the original instruction, which is the corresponding condition bit in the CFLAGS register that will be set or referenced.

Taking the translation from ARM to RISC-V as an example, the dynamic execution mode on RISC-V provides the hardware running environment for the translated target binary program. First of all, the condition bit register CFLAGS maps the condition bits of

ARM one by one, so that the target binary program does not need extra storage to simulate the condition bits when running, thereby reducing the memory access operation. Secondly, the CCR instruction is used to indicate whether the successor instruction following it sets or references the condition bit register CFLAGS. Thirdly, aiming at the arithmetic instruction for ARM through setting or referencing the condition bits, we extended the corresponding RISC-V arithmetic instruction function with the setting or referencing of the condition bit so that the translated setting and referencing instructions for the condition bit do not need additional calculation operations, thereby improving the efficiency of the dynamic execution. Finally, the IR was designed according to the source ISA in the dynamic translator to match and keep the information of the source program instruction sequence for the condition bit operation as far as possible. Thus, the translation does not need complex data flow graph analysis and pattern matching algorithms within and between the basic blocks, and direct instruction mapping is adopted, which greatly reduces the translation complexity. For conditional codes implicitly referencing the condition bits in the ARMV8 A32 instruction code, we can also identify them by CCR instructions during translation, thus archiving rapid translation to the RISC-V instruction, and greatly simplifying the translation complexity and translated instruction sequence.

As shown in Figure 2, the hardware non-invasive mapping method for condition bits for binary translation proposed in this paper includes an execution mode switching mechanism, a condition bit register CFLAGS, the identifier instruction for the condition bit, extension instructions for the setting of the condition bit, and an extension of the instructions for the referencing of the condition bit and the dynamic translator.

3.2. Execution Mode Switching Mechanism

The execution mode switching mechanism is responsible for switching back and forth between the translation mode and the dynamic execution mode. As shown in Figure 3, in order to be compatible with the original programming model and not modify the original instruction code, we used the target address least significant bit of the indirect jump instruction in the target ISA to identify mode switching. When the least significant bit of the target address of the indirect jump instruction is '1', the target processor enters or stays in the dynamic execution mode, and can access the condition bit register, extension instructions, etc. Meanwhile, when the least significant bit of the target address of the indirect jump instruction is '0', the target processor enters or remains in the translation mode.

After the dynamic translator completes a section of code translation, the system needs to jump from the translator to the translated target binary program block. At this time, the target address least significant bit of the indirect jump instruction is set to '1', so as to achieve the switch from the translation mode to the dynamic execution mode. When the target processor is found at the untranslated code during executing the target binary program block, it needs to be switched to the translator for the translation task. At this time, the target address least significant bit of the indirect jump instruction is cleared to '0', so as to achieve the switch from the dynamic execution mode to the translation mode. When the target processor is started, it runs in the translation mode at first.

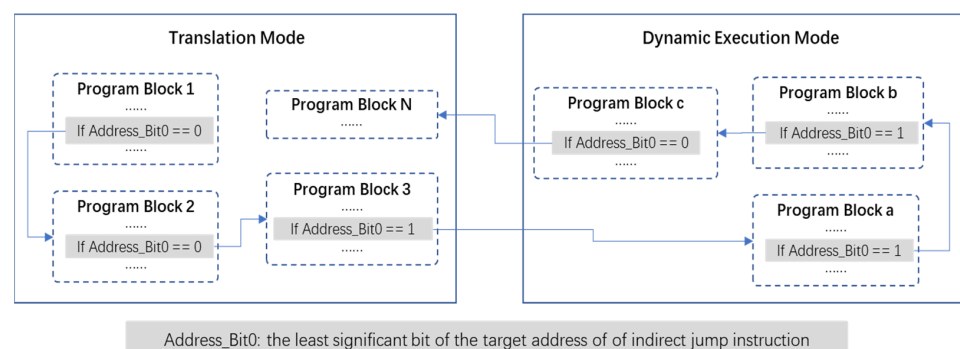


Figure 3. The HNIMCB execution mode.

3.3. Condition Bit Register (CFLAGS Register)

This paper took ARMV8 and RISC-V as an example to design condition bit mapping between the source and the target ISA. In the ARM ISA, there are NZCV condition bits in the PSTATE register, and arithmetic, comparison, or MSR instructions can potentially set or reference these condition bits. However, there are no condition bits in RISC-V, and the setting and referencing of the condition bits are directly implemented within a single branch instruction. This difference makes the translation from the ARM's setting and referencing instructions of the condition bits to RISC-V instructions requiring a lot of computation instructions and memory access operations.

As shown in Table 2, this paper designs and implements the condition bit register (CFLAGS) in the dynamic execution mode for RISC-V. The condition bits in the register directly map to the condition bits of ARM. C_S is the negative status flag bit corresponding to the ARM N bit, C_Z is the 0-status flag bit corresponding to the ARM Z bit, C_C is the carry status flag bit corresponding to the ARM C bit, and C_O is the overflow flag bit corresponding to the ARM V bit.

Table 2. Mapping NZCV to the CFLAGAS.

ARM PSTATE	CFLAGS	Description
N	C_S	Negative status flag bit: when the signed number represented by two complement codes is used for the operation, N = 1 indicates that the result of the operation is negative; N = 0 indicates that the result of the operation is either positive or zero.
Z	C_Z	Zero status flag bit: Z = 1 indicates that the result of the operation is zero, while Z = 0 indicates that the result of the operation is non-zero, respectively.
C	C_C	Carry status flag bit: there are four ways to set the value of C. Addition operation (including CMN): when the operation result produces a carry (unsigned number overflow), C = 1, otherwise C = 0; Subtraction operation (including CMP): when the operation generates a borrow (unsigned number overflow), C = 0, otherwise C = 1; For non-add/subtract instructions that include shift operations, C is the last bit of the shift out value; for other non-add/subtract instructions, the value of C does not usually change.
V	C_O	Overflow status flag bit: V can be set in two ways: for the addition and subtraction operation instruction, when the operand and the operation result are signed numbers represented by the binary complement, V = 1 indicates that the sign bit is overflow; for other non-add/subtract instructions, the value of V does not usually change.

3.4. Identifier Instruction for the Condition Bits (CCR Instruction)

As there are no condition bits in RISC-V, the setting and referencing of the condition bits are directly implemented within a single branch instruction [4,5], and arithmetic instructions do not set or reference the condition bits. As shown in Table 3, in order to be compatible with the original programming model of RISC-V, this paper proposes the extension of the identifier instruction for condition bit (CCR Instruction), which can be combined with the succeeding instruction to set or reference the condition bits based on the identification specified by the CCR instruction.

When the extended RISC-V executes the CCR instruction in the dynamic execution mode, the hardware firstly decodes the CO [0:2], COND [3:6], and CC [7:10] operand information encoded in the CCR instruction. During the succeeding instruction execution, the target processor hardware determines the appropriate setting and referencing of the condition bit based on the CCR operand information and the value of the condition bit in the CFLAGS register.

Table 3. The encoding fields of CCR instruction.

Field	Semantics	Value (Mnemonic)	Description
CO [0:2]	Condition bits operations	000 (co_none)	The succeeding instruction does not set or refer to condition bits.
		001 (co_set)	The succeeding instruction sets condition bits.
		010 (co_ref)	The succeeding instruction refers to condition bits.
		011 (co_sr)	The succeeding instruction sets and refers to condition bits.
		110 (co_cond)	The succeeding instruction sets condition bits with COND [3:6].
		xxx (co_res)	Reserved.
COND [3:6]	Condition bits immediate	0000-1111	Condition bits encoded in the source instruction, such as the NZCV field in the ARM CCMP instruction.
CC [7:10]	Conditional execution code	0000-1111	Condition codes encoded in the source instruction, such as the COND field in the ARM CCMP instruction.

3.5. Translation of the Setting of the Condition Bits

In the ARM ISA, arithmetic instructions, comparison, or MSR instructions can potentially set or reference the condition bits. While in the RISC-V ISA, there are no condition bits, and the setting and referencing of the condition bits are directly implemented within a single instruction [4,5]. This can lead to a low efficiency of translation from the source ISA to the target ISA or require more redundant computation. A more detailed instructions list and the condition bits setting method have been depicted in Table 4.

As shown in Table 4, in the dynamic execution mode, this paper extended the functionality of the RISC-V arithmetic instructions. A pair of RISC-V arithmetic instruction and CCR instructions complete the translation of the ARM instruction which sets the condition bits.

As shown in Figure 4, taking ARM ADDS instruction as an example, in order to make the function of the RISC-V ADD instruction consistent with the ARM ADDS instruction, the setting of the condition bits in the CFLAGS register is added through the CCR instruction.

Table 4. The mapping table for the setting of the condition bits.

ARM Instruction	Semantics	Description	Setting of the Condition Bits	Translated Instructions
ADDS	$rd = rn + op2$	Add (ext register/imm/shifted register)	$PSTATE<N, Z, C, V> =$ $result<datasize-1>:$ $IsZeroBit(result):$	<code>ccr 0, cc_al, co_set</code> <code>add rd, rn, op2</code>
ADCS	$rd = rn + rm + C$	Add with Carry	$IsZeroBit(result):$ $IsUnsignedOverflow():$ $IsSignedOverflow();$	<code>ccr 0, cc_al, co_sr</code> <code>add rd, rn, op2</code>
CMN	$rn + op2$	Compare Negative (ext register/imm/shifted register), an alias of ADDS		<code>ccr 0, cc_al, co_set</code> <code>add x0, rn, op2</code>
SUBS	$rd = rn - op2$	Subtract (ext register/imm/shifted register)		<code>ccr 0, cc_al, co_set</code> <code>sub rd, rn, op2</code>
CMP	$rn - op2$	Compare (ext register/imm/shifted register), an alias of SUBS		<code>ccr 0, cc_al, co_set</code> <code>sub x0, rn, op2</code>
NEGS	$rd = -op2$	Negate, an alias of SUBS	$PSTATE<N, Z, C, V> =$ $result<datasize-1>:$ $IsZeroBit(result):$ $!IsUnsignedOverflow():$ $IsSignedOverflow();$	<code>ccr 0, cc_al, co_set</code> <code>sub rd, x0, op2</code>
SBCS	$rd = rn - op2 - \sim C$	Subtract with Carry		<code>ccr 0, cc_al, co_sr</code> <code>sub rd, rn, op2</code>
NGCS	$rd = -rn - \sim C$	Negate with Carry, an alias of SBCS		<code>ccr 0, cc_al, co_sr</code> <code>sub rd, x0, op2</code>
SUBPS	$rd = rn - rm$	Subtract Pointer		<code>ccr 0, cc_al, co_set</code> <code>sub rd, rn, rm</code>
ANDS	$rd = rn \& op2$	Bitwise AND (imm/shifted register)	$PSTATE<N, Z, C, V> =$ $result<datasize-1>:$ $IsZeroBit(result):$	<code>ccr 0, cc_al, co_set</code> <code>and rd, rn, op2</code>
BICS	$rd = rn \& \sim op2$	Bitwise Bit Clear (shifted register)		<code>not rt, op2</code>
TST	$rn \& op2$	Test bits (imm/shifted register), an alias of ANDS	'00'	<code>ccr 0, cc_al, co_set</code> <code>and x0, rn, op2</code>

Table 4. Cont.

ARM Instruction	Semantics	Description	Setting of the Condition Bits	Translated Instructions
CCMN	if (cc) rn + op2 else N:Z:C:V = opcode(nzcv)	Conditional Compare(imm/register)	if(cc) PSTATE<N, Z, C, V> same to cmn or cmp	ccr nzcv, cc, co_cond add x0, rn, op2
CCMP	if (cc) rn − op2 else N:Z:C:V = opcode(nzcv)	Conditional Compare Negative(imm/register)	else PSTATE<N, Z, C, V> = opcode(nzcv)	ccr nzcv, cc, co_cond sub x0, rn, op2
MSR	sysreg = Xn	Move value to Special Register	PSTATE<N, Z, C, V> = Xn<31:28>	csrw cflags, xn

Encoding of RISC-V ADD

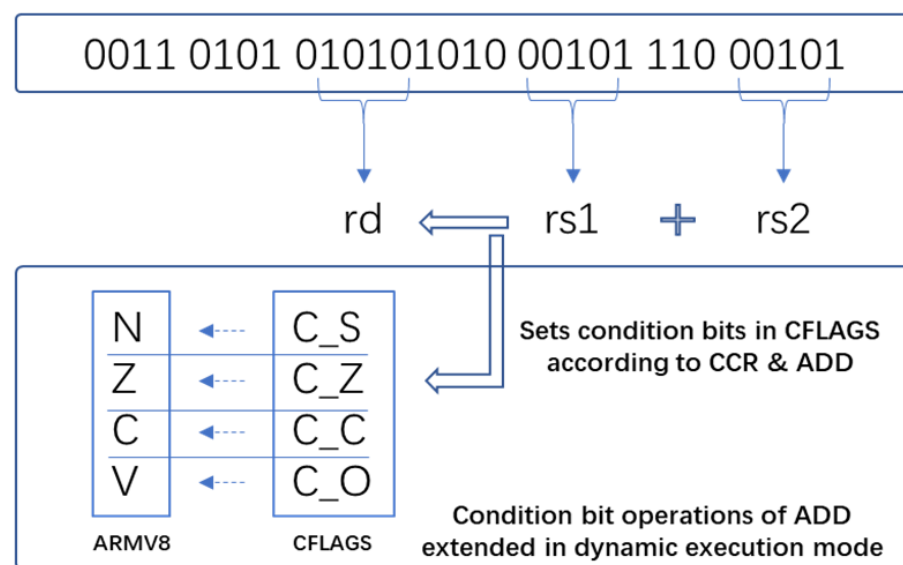


Figure 4. Condition bits setting in the HNIMCB.

3.6. Translation of the Referencing of the Condition Bits

In the ARM ISA, some arithmetic instructions, branch instructions, or MRS may refer to the condition bits, such as carry-addition, borrow-subtraction, and other arithmetic instructions, as well as conditional jump instructions. However, the RISC-V does not have condition bits, and the setting and referencing of the condition bits are directly implemented in one instruction [4,5]. This can cause the generation of a low efficiency of translation from the ARM to the RISC-V or lead to the requiring of more redundant calculations. A more detailed instructions list and the condition bits referencing method have been depicted in Table 5.

As shown in Table 5, in the dynamic execution mode, this paper extended the corresponding arithmetic instructions in RISC-V for the referencing instructions of the condition bit in the ARM ISA. By combining the identifier instruction of the condition bit, a one-to-one mapping relationship is thereby achieved from the ARM to the RISC-V.

As shown in Figure 5, taking the ARM ADC instruction as an example, the RISC-V ADD instruction corresponds to its functionality. In order to make the RISC-V ADD instruction function consistent with that of the ARM ADC instruction, on the basis of the original $rd = rs1 + rs2$ function of ADD, the referencing of the condition bits in the CFLAGS register is added through the CCR instruction, so that the function is directly mapped with the ARM ADC instruction.

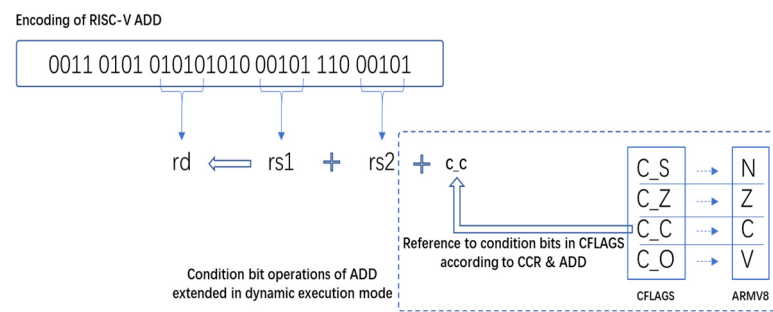


Figure 5. Condition bits referencing in the HNIMCB.

Table 5. The mapping table for the referencing of the condition bits.

ARM Instruction	Semantics	Description	Translated Instructions
ADC	$rd = rn + rm + C$	Add with Carry	<pre> ccr 0, cc_al, co_ref add rd, rn, op2 </pre>
SBC	$rd = rn - op2 - \sim C$	Subtract with Carry	<pre> ccr 0, cc_al, co_ref sub rd, rn, op2 </pre>
NGC	$rd = -rm - \sim C$	Negate with Carry, an alias of SBC.S	<pre> ccr 0, cc_al, co_ref sub rd, x0, op2 mv rd, rm </pre>
CSEL	if(cc) $rd = rn$; else $rd = rm$	Conditional Select	<pre> ccr 0, cc, co_none mv rd, rn li rd, 0 </pre>
CSET	if(cc) $rd = 1$; else $rd = 0$	Conditional Set: an alias of CSINC	<pre> ccr 0, cc, co_none li rd, 1 li rd, 0 </pre>
CSETM	if(cc) $rd = \sim 0$; else $rd = 0$	Conditional Set Mask: an alias of CSINV	<pre> ccr 0, cc, co_none not rd, x0 addi rd, rm, 1 </pre>
CSINC	if(cc) $rd = rn$; else $rd = rm + 1$	Conditional Select Increment	<pre> ccr 0, cc, co_none mv rd, rn not rd, rm </pre>
CSINV	if(cc) $rd = rn$; else $rd = \sim rm$	Conditional Select Invert	<pre> ccr 0, cc, co_none mv rd, rn neg rd, rm </pre>
CSNEG	if(cc) $rd = rn$; else $rd = -rm$	Conditional Select Negate	<pre> ccr 0, cc, co_none mv rd, rn mv rd, rm </pre>
CINC	if(cc) $rd = rn + 1$; else $rd = rn$	Conditional Select Increment	<pre> ccr 0, cc, co_none addi rd, rd, 1 mv rd, rn </pre>
CINV	if(cc) $rd = \sim rn$; else $rd = rn$	Conditional Invert: an alias off CSINV	<pre> ccr 0, cc, co_none not rd, rd mv rd, rn </pre>
CNEG	if(cc) $rd = -rn$; else $rd = rn$	Conditional Negate: an alias of CSNEG	<pre> ccr 0, cc, co_none neg rd, rd </pre>
B.cond	if(cc) $PC = PC + \text{offset}$	Branch conditionally	<pre> ccr 0, cc, co_none jal x0, offset </pre>
CCMN	if (cc) $rn + op2$ else $N:Z:C:V = \text{opcode}(nzcvc)$	Conditional Compare(imm/register)	<pre> ccr nzcvc, cc, co_cond add x0, rn, op2 </pre>
CCMP	if (cc) $rn - op2$ else $N:Z:C:V = \text{opcode}(nzcvc)$	Conditional Compare Negative(imm/register)	<pre> ccr nzcvc, cc, co_cond sub x0, rn, op2 </pre>
MRS	$Xn = \text{sysreg}$	Move value from Special Register to gr	<pre> csrr xn, cflags </pre>

Figure 6 shows an example of the translation from the ARM to RISC-V with an extension for the CCR instruction and arithmetic instruction extended for the condition bit setting or referencing. The subsequent target instruction sequence of the translation mapping is much simpler after the extension compared to before.

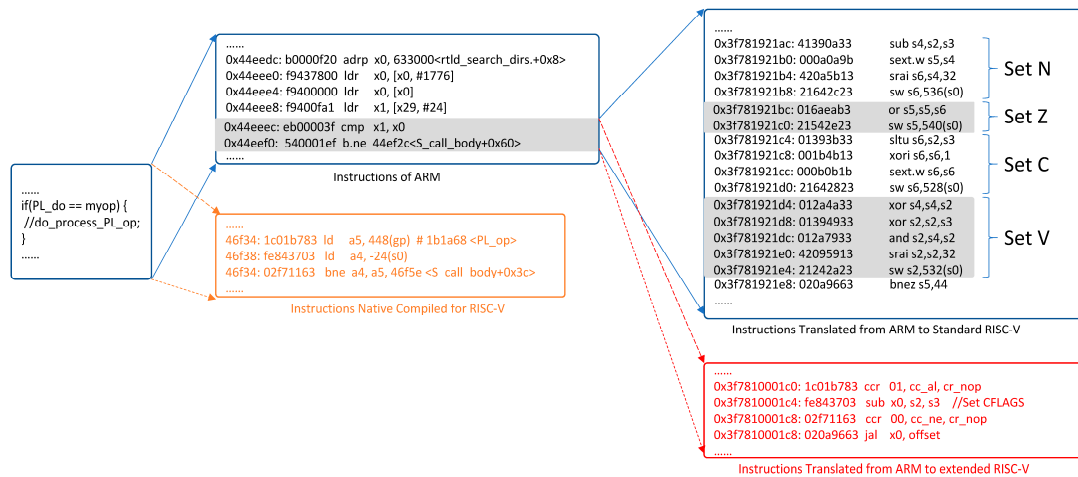


Figure 6. The HNIMCB vs. the standard RISC-V in translation.

3.7. Dynamic Translator

Binary translation technology includes static translation and dynamic translation. Due to the limitations of information in source binary programs, static translation technology cannot solve all the problems, such as indirect jumps, code and data mixing, self-generated codes, and so on. The translator implemented in this paper adopted the dynamic translation mode.

This translator has its own intermediate representation (IR) and consists of a front end and a back end. The front end converts the binary code of the source ISA to IR, and the back end implements the mapping of the IR to the target ISA. The design of the IR is crucial in compilation technology, as it determines how much information is provided for the back-end translation and optimization. As shown in Figure 7, taking the ARM ADDS instruction as an example, if the design of the IR refers to the target ISA rather than the source ISA, it is possible that ADDS can be transformed into multiple independent IR representations during front-end conversion. In this case, the back end of the translator needs to implement a complex pattern-matching algorithm to reintegrate these multiple IR representations into a single back-end instruction description, greatly increasing the complexity of translation.

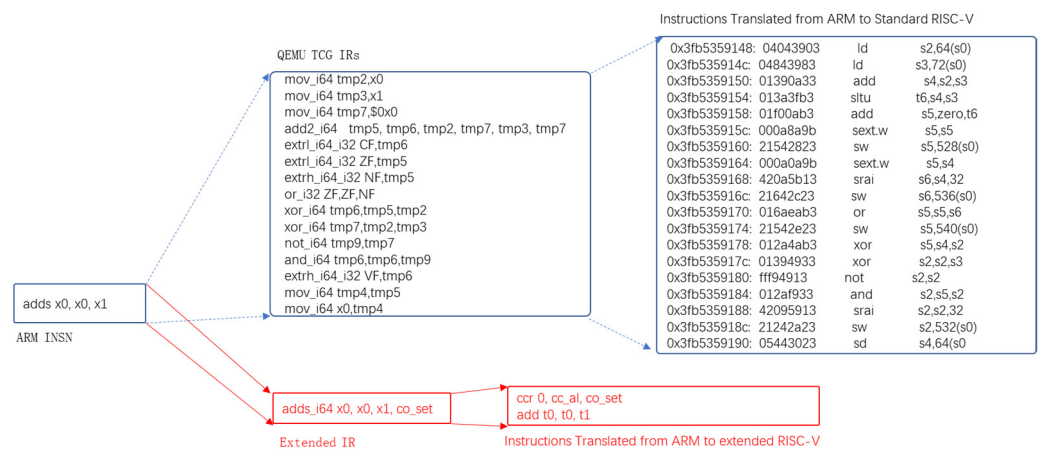


Figure 7. Mapping from the source instruction to IR.

To reduce the complexity of data flow analysis and pattern-matching algorithms, the dynamic translator employed in this paper defined many new IRs that are close to the source ISA. If the source instruction includes a condition code, such as the CCMP instruction, the condition code will also be encoded as an immediate parameter to the

IR. This allows the front end of the translator to preserve the information of the original instructions as much as possible when translating them into IR. When converting the IR into the target ISA in the back end of a translator, the original information of the source instruction can be obtained as much as possible, thereby allowing for a one-to-one mapping with extended resources or instructions in dynamic execution mode. This simplifies the analysis of data flow and control flow, reduces the fusion of multiple IRs, and thus reduces the translation complexity.

In the conversion of the IRs to the back-end RISC-V instructions, usually a CCR instruction will be emitted first according to the IR semantics. As shown in Figure 7, the 'adds_i64' IR mapping to the ARM ADDS instruction will emit a 'ccr 0, cc_al, co_set' instruction, which combines with the RISC-V ADD instruction to achieve the function of setting the CFLAGS register like the ADDS in the ARM ISA.

3.8. Optimization of Translation Efficiency

The efficiency of binary translation is measured by the additional time consumption T_{total} during the process of translation and execution of the program, including the time consumption of the translation process itself $T_{translate}$, as well as the difference between the execution time of the translated program T_{run} and the execution time of the same program natively $T_{run_in_native}$, as shown in Formula (1):

$$T_{total} = T_{translate} + (T_{run} - T_{run_in_native}), \quad (1)$$

Binary translators pay more attention to $(T_{run} - T_{run_in_native})$, ignoring $T_{translate}$, resulting in increasingly complex translation algorithms. For example, pattern matching is used to reduce the number of setting and referencing instructions for the condition bits, and data flow analysis is used to achieve optimization methods, such as deleting redundant condition bit calculations. The complexity of these algorithms is directly related to the number of basic blocks. Assuming that each basic block has 2 branches and there are n layers of successor basic blocks, the calculation complexity is $O(2^n)$ [22]. By extending the hardware functions for the condition bit of the arithmetic instructions in the target ISA and the corresponding IR descriptions in the translator, this paper thereby achieved the efficient mapping from the source instructions to the target instructions, which eliminates the data flow analysis process, simplifies instruction pattern matching, and greatly reduces the complexity of dynamic translation. Additionally, there is no need for additional information storage when executing the translated code.

Currently, binary translators mostly use delayed calculation to reduce the calculation of the condition bits in translation optimization. However, delayed calculations increases the memory access consumption during dynamic execution, as it needs to save instruction codes, operands, and other information [21]. This paper expands the condition bit register "CFLAGS", and the function of the setting and referencing instruction for the condition bits in the target ISA so that it achieves a one-to-one mapping of the condition bit instructions between the source and the target ISA. As a result, there is no need for delayed calculations to obtain the value and save of the condition bits, eliminating the memory access operation caused by the delayed calculation of the condition bits. As shown in Figure 6, the one-to-one mapping of the condition bit instructions between the source and target ISA greatly eliminates additional calculations for the condition bits and the memory access operation for simulating the condition bits.

In summary, the hardware non-invasive mapping method for the condition bits in this paper eliminates the process of pattern matching and data flow analysis, eliminates the storage of the instructions/operands for delayed calculations, and further eliminates the additional operations, such as the calculations of the condition bit and memory simulation for the condition bit, so as to significantly improve the efficiency of binary translation.

4. Results

4.1. Experiment Platform

As shown in Figure 8, this paper adopts the QEMU system mode implemented on an x86 PC to simulate the hardware experimental platform of RISC-V processors, which we refer to as the QEMU-System-RV64. Thus, we can conveniently expand the capabilities of the condition bit register CFLAGS, the setting, and the referencing of the condition bits for RISC-V processors on the QEMU-System-RV64. The experiments were conducted by running the QEMU user mode on the QEMU-System-RV64, serving as the architecture basis for the dynamic translation from the ARM to RISC-V. We inherited the QEMU's framework, including the IR definition, translation optimization, and other capabilities, while implementing the front end for the ARM ISA and the back end for the RISC-V standard ISA. This forms the binary translator from ARM to RISC-V called the QEMU-User-ARM running on the QEMU-System-RV64.

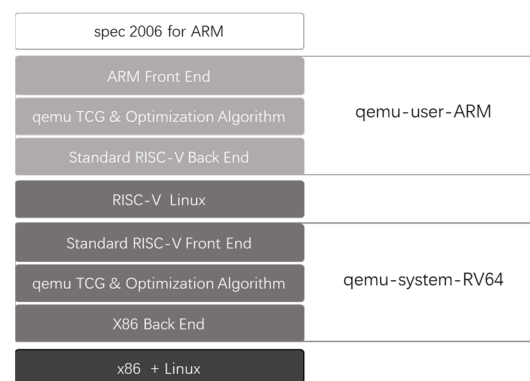


Figure 8. The standard experimental platform.

As shown in Figure 9, according to the description in Section 3, this experiment extended the execution mode switching mechanism, CCR instruction, CFLAGS register, and the setting or referencing instructions of the condition bits for RISC-V simulated in the QEMU-System-RV64 to obtain the extended RISC-V simulator named the QEMU-System-RV64-ext. Following the description in Section 3, this experiment extended the IR definition and the translation back-end for CFLAGS, CCR instruction, and the setting or referencing instructions of the condition bits based on the QEMU-User-ARM. This forms a binary translator from ARM to an extended RISC-V processor termed the QEMU-User-ARM-ext.

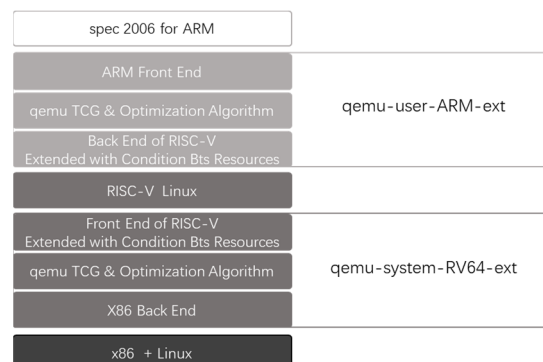


Figure 9. The HNIMCB experimental platform.

4.2. Experiment Steps

The experiment used SPEC 2006 as the test benchmark with a GCC compiler and an O3 compilation option, and the test process is divided into three steps. Firstly, a standard test platform was built using the QEMU-System-RV64 and QEMU-User-ARM. When

the ARM binary programs were translated and executed, the QEMU-System-RV64 then recorded the number of translated RISC-V instructions and memory accesses, while the QEMU-User-ARM records the number of condition bit instructions in the ARM binary program. Secondly, an extended test platform was built using QEMU-System-RV64-ext and QEMU-User-ARM-ext. When the ARM binary programs were translated and executed, the QEMU-System-RV64-ext recorded the number of translated and executed RISC-V instructions and memory accesses, while the QEMU-User-ARM-ext recorded the number of condition bit instructions in the ARM binary program. Finally, we compared and analyzed the data obtained in the first two steps, such as the number of dynamic running instructions, the number of memory access instructions, and the number of condition bit instructions.

4.3. Total Instruction Statistics

Dynamic instruction count is one of the important performance indicators in binary translation. This paper specifically conducted statistical analysis on this indicator for assessing the SPEC 2006 benchmark. Each data in the bar charts displayed in Figure 10 represents the percentage decrease in the total dynamic instruction count of different applications before and after applying the hardware non-invasive mapping method for condition bits proposed for binary translation.

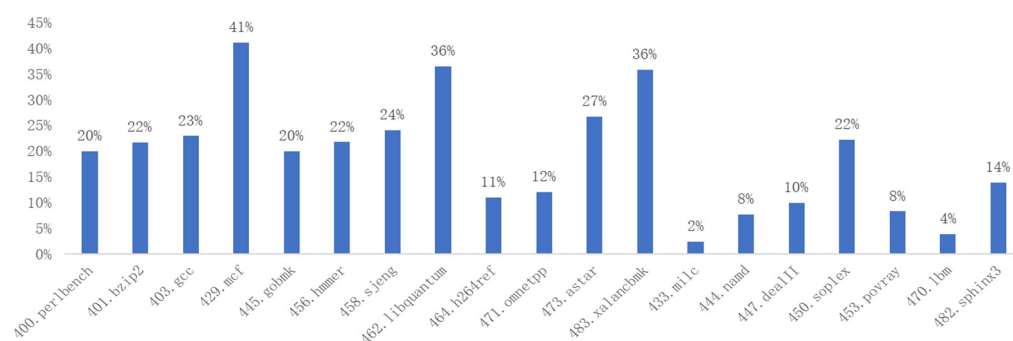


Figure 10. Total instruction reduction.

The data indicate that after optimization with the hardware non-invasive mapping method for condition bits proposed in this paper, the total dynamic instruction count decreased by up to 41%, and by 19% on average, respectively. The data also indicates that most programs have a significant optimization effect, mostly because the ARM condition bit instructions are mapped to significantly fewer RISC-V instructions when translated using the proposed method. More details can be found in Figure 6. In summary, the experimental data shows that the proposed method effectively decreases the total dynamic instruction count in the translated programs.

4.4. Memory Access Statistics

Optimization of memory access instruction count is a topic that has been widely discussed in binary translation, specifically in relation to register allocation or condition bit mapping. This paper presents the experimental results on the number of memory access instructions in the SPEC 2006 program after binary translation from ARM to RISC-V. Each data point in the bar chart shown in Figure 11 represents the percentage decrease in the memory access instructions before and after applying the hardware non-invasive mapping method for condition bits proposed for binary translation.

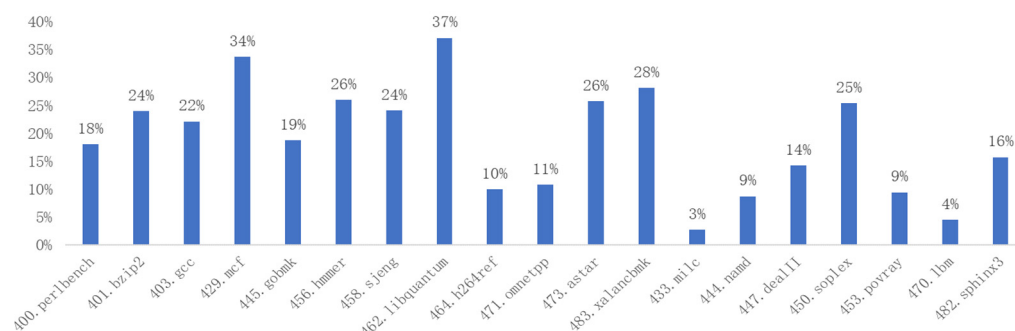


Figure 11. Memory access reduction.

The data shows that after using the optimization method proposed in this paper, the number of memory access instructions during the dynamic runtime of binary translation decreases by up to 37%, and by 19% on average, respectively. The experimental data also shows that most of the programs were significantly optimized, mainly because the ARM NZCV condition bits will be directly mapped by the CFLAGS register, instead of mapped by a memory unit. To sum up, the experiment data shows that the proposed method effectively decreases the number of memory access in translated programs.

4.5. Regression Analysis

In order to further analyze the effectiveness of the proposed method, this paper used the linear regression analysis method to analyze the relationship between the efficiency of memory access optimization and the proportion of condition bit instructions. Figure 12 shows a scatter plot and regression analysis of the proportion of memory access instructions reduction in the SPEC 2006 benchmark and the proportion of condition bit instructions in the source binary programs.

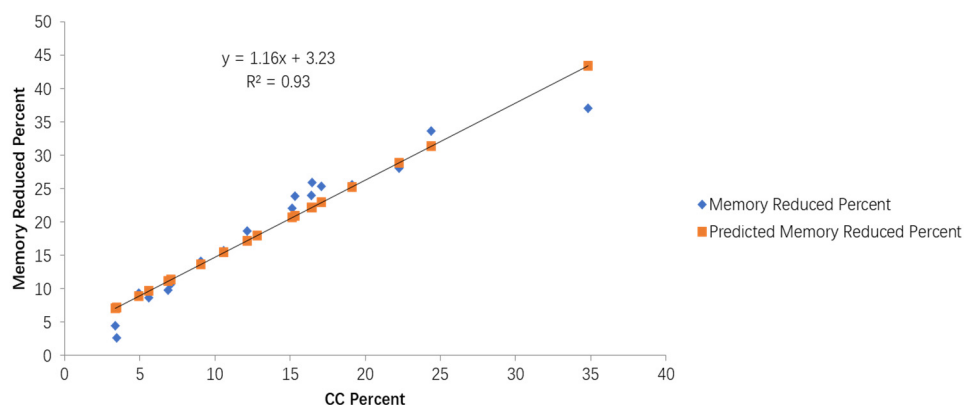


Figure 12. Regression analysis of memory reduction.

According to Figure 12, after optimizing with the hardware non-invasive mapping method for condition bits proposed for binary translation, the proportion of memory access instructions reduction was similar to the predicted value calculated by the linear regression equation, and the obtained R^2 was 0.93, which is a good fit of the model. This means that the proportion of condition bit instructions has a positive linear correlation with the reduction in the memory access instructions. In other words, the higher the proportion of condition bit instructions, the greater the reduction in the memory access instructions. Using the same analysis method, it can be concluded that there is a similar linear relationship between the proportion of total instructions reduction and the proportion of condition bit instructions in the source program.

5. Discussion

5.1. Microarchitecture Impact

It is true that most binary translation techniques aim to execute code in an unmodified target architecture, but there is a performance bottleneck that needs to be addressed. This paper specifically addresses the issue of optimizing the translation of condition bit operations. Firstly, RISC-V is an open-source and open architecture that allows for specific issues to be addressed by defining sub-extensions. Secondly, existing open-source binary translators generally have a low translation efficiency, while binary translators with a higher efficiency are developed through architecture customization by commercial companies, such as Apple's Rosetta 2. When translating X86 programs to ARM, Rosetta 2 provides resources to ensure TSO memory ordering [25,26], among others.

Our extensions enhance the semantics of some instructions, but overall, the instructions are still relatively simple, and simpler than the ARM instructions. For instance, ADD instructions with the HNIMCB does not require a complicated operation, such as the handling of source register extensions and shifts [27] in ARM.

The operands of the CCR instruction are all immediate values, meaning that the following instructions know that they need to operate on the CFLAGS without waiting for the CCR instruction to execute. In this manner, the CCR instruction does not need to establish any dependency with the subsequent ALU instructions. After the decoding stage, the CCR instruction can be executed as a NOP.

Another similar case of CCR is the VSETVL instruction in the RISC-V vector extension [28], which has been ratified, and many CPU cores (such as the C908 [29] for T-HEAD, and P270 [30] from Sifive) have delivered this extension. In the vector extension, the vsetvl instruction will set the VL and VTYPE CSR. It has been used very frequently for many workloads. For example, some test cases of OpenCV, the rate of vsetvl will be more than 20% of the total vector instructions. With pipeline forwarding and register renaming, it will not be a big challenge for the latency [31].

Thus, we expect that our approach will not have a significant impact on the pipeline performance. As our experiment platform is QEMU, a function level simulator, we cannot provide a quantitative analysis on this question at this time, and this issue needs further research to explore.

5.2. X86 Translation

This article proposed a method of establishing general condition bit semantics on RISC-V. It does not simulate concrete condition bit instructions. Instead, it focuses on the abstract condition bit operation mode, such as the condition bit setting, referencing, and the conditional execution codes.

Although ARM has been used throughout the entire article and experimental part, this mechanism can also be applied to the translation from other architectures, such as X86 to RISC-V. X86 has six status flags (CF, OF, SF, ZF, and PF, respectively) in the EFLAGS register [32], which can be easily mapped by the proposed CFLAGS register. These status flags will be set by addition, subtraction, comparison, and other instructions. The CMOVcc, FCMOVcc, Jcc, and SETcc assess the condition codes, encoded by the one or more status flags. The proposed CCR instruction encodes both the setting and referencing of the condition bits and the condition codes. With the CCR instruction, various condition bit operations from other architectures can be mapped to the most basic RISC-V instructions, such as addition, subtraction, and logical instructions.

6. Conclusions

This paper presents a statistical analysis of the percentage of condition bit instructions in the SPEC 2006 Benchmark and analyzes the condition bit instruction translation techniques of the existing mainstream binary translation tools. Based on previous research, this paper proposes a hardware non-invasive mapping method for the translation of condition bit instructions in the RISC-V open-source architecture. This method extends the

RISC-V dynamic execution mode to implement hardware functions and resources, such as the condition bit register, and the setting or referencing of the condition bits, enabling the efficient translation of the condition bit instructions from the source instruction set to RISC-V, while ensuring RISC-V compatibility with the existing software ecosystem. The experimental data show that the proposed method reduces the total number of instructions by up to 41%, and the number of memory access instructions by up to 37%, respectively, effectively reducing the translation complexity and improving the translation performance.

RISC-V has achieved a significant technological and commercial development due to its open-source and open ecosystem. However, its software and application ecosystem in high-performance areas like in mobile devices, desktops, and data centers falls short. The proposed hardware non-invasive mapping method for condition bits can effectively optimize the binary translation performance from ARM or x86 to RISC-V, and further promotes the commercialization process of RISC-V across high-performance fields.

Author Contributions: Conceptualization, C.L.; methodology, C.L. and Z.L.; software, Z.L. and Y.S.; validation, Z.L., L.H., Y.S. and X.Y.; project administration, C.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The authors confirm that the data supporting the findings of this study are available within the article.

Conflicts of Interest: The authors declare no conflict of interest.

References

- Barham, P.; Dragovic, B.; Fraser, K.; Hand, S.; Harris, T.; Ho, A.; Neugebauer, R.; Pratt, I.; Warfield, A. Xen and the Art of Virtualization. In Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles, New York, NY, USA, 19 October 2003; ACM: New York, NY, USA; pp. 164–177.
- Li, H.; Xu, X.; Ren, J.; Dong, Y. ACRN: A Big Little Hypervisor for IoT Development. In Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, Providence, RI, USA, 14 April 2019; ACM: Providence, RI, USA; pp. 31–44.
- Li, C.; Guo, R.; Tian, X.; Wang, H. KHV: KVM-Based Heterogeneous Virtualization. *Electronics* **2022**, *11*, 2631. [\[CrossRef\]](#)
- Waterman, A.; Asanovic, K. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*; Document Version 20191213; EECS Department, University of California: Los Angeles, CA, USA, 2019.
- Waterman, A.; Asanovic, K. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*; Document Version 20190608-Priv-MSU-Ratified; EECS Department, University of California: Los Angeles, CA, USA, 2019.
- Chernoff, A.; Herdeg, M.; Hookway, R.; Reeve, C.; Rubin, N.; Tye, T.; Bharadwaj Yadavalli, S.; Yates, J. FX!32 a Profile-Directed Binary Translator. *IEEE Micro* **1998**, *18*, 56–64. [\[CrossRef\]](#)
- Baraz, L.; Devor, T.; Etzion, O.; Goldenberg, S.; Skaletsky, A.; Wang, Y.; Zemach, Y. IA-32 Execution Layer: A Two-Phase Dynamic Translator Designed to Support IA-32 Applications on Itanium/Spl Reg/-Based Systems. In Proceedings of the 22nd Digital Avionics Systems Conference, Proceedings (Cat. No.03CH37449), San Diego, CA, USA, 5 December 2003; IEEE Comput. Soc.: San Diego, CA, USA, 2003; pp. 191–201.
- Cifuentes, C.; Van Emmerik, M. UQBT: Adaptable Binary Translation at Low Cost. *Computer* **2000**, *33*, 60–66. [\[CrossRef\]](#)
- Dehnert, J.C.; Grant, B.K.; Banning, J.P.; Johnson, R.; Kistler, T.; Klaiber, A.; Mattson, J. The Transmeta Code Morphing/Spl Trade/Software: Using Speculation, Recovery, and Adaptive Retranslation to Address Real-Life Challenges. In Proceedings of the International Symposium on Code Generation and Optimization, CGO 2003, San Francisco, CA, USA, 23–26 March 2003; IEEE Comput. Soc.: San Francisco, CA, USA, 2003; pp. 15–24.
- Bellard, F. *QEMU, a Fast and Portable Dynamic Translator*; USENIX Association: Anaheim, CA, USA, 2005; p. 41.
- Box86. Available online: <https://github.com/ptitSeb/box86> (accessed on 26 May 2023).
- Houdek, R. FEX-Emu: Fast(-Er) X86 Emulation for AArch64. In Proceedings of the Free and Open source Software Developers' European Meeting (FOSDEM), online, 5–6 February 2022.
- Engelke, A.; Schulz, M. Instrew: Leveraging LLVM for High Performance Dynamic Binary Instrumentation. In Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, Lausanne, Switzerland, 17 March 2020; ACM: Lausanne, Switzerland; pp. 172–184.
- Liao, Y.; Sun, G.; Jiang, H.; Jin, G.; Chen, G. All Registers Direct Mapping Method in Dynamic Binary Translation. *Comput. Appl. Softw.* **2011**, *28*, 21–24+48. [\[CrossRef\]](#)
- Wang, J.; Pang, J.; Fu, L.; Yue, F.; Shan, Z.; Zhang, J. A Dynamic and Static Combined Register Mapping Method in Binary Translation. *J. Comput. Res. Dev.* **2019**, *56*, 708–718. [\[CrossRef\]](#)

16. Smelyanskiy, M.; Tyson, G.S.; Davidson, E.S. Register Queues: A New Hardware/Software Approach to Efficient Software Pipelining. In Proceedings of the 2000 International Conference on Parallel Architectures and Compilation Techniques (Cat. No.PR00622), Philadelphia, PA, USA, 15–19 October 2000; IEEE Comput. Soc.: Philadelphia, PA, USA, 2000; pp. 3–12.
17. Wang, C.; Wu, Y.; Rong, H.; Park, H. SMARQ: Software-Managed Alias Register Queue for Dynamic Optimizations. In Proceedings of the 2012 45th Annual IEEE/ACM International Symposium on Microarchitecture, Vancouver, BC, Canada, 1–5 December 2012; IEEE: Vancouver, BC, Canada, 2012; pp. 425–436.
18. Wen, Y.; Tang, D.; Qi, F. Register Mapping and Register Function Cutting out Implementation in Binary Translation. *J. Softw.* **2009**, *20*, 1–7.
19. Lattner, C.; Adve, V. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In Proceedings of the International Symposium on Code Generation and Optimization, CGO 2004, San Jose, CA, USA, 20–24 March 2004; IEEE: San Jose, CA, USA, 2004; pp. 75–86.
20. Mac Benchmarks. Available online: <https://browser.geekbench.com/mac-benchmarks> (accessed on 26 May 2023).
21. Ma, X.; Wu, C.; Tang, F.; Feng, X.; Zhang, Z. Two Condition Code Optimization Approaches in Binary Translation. *J. Comput. Res. Dev.* **2005**, *42*, 329–337. [\[CrossRef\]](#)
22. Tang, F.; Wu, C.; Feng, X.; Zhang, Z. EFLA Algorithm Based on Dynamic Feedback. *J. Softw.* **2007**, *18*, 1603–1611. [\[CrossRef\]](#)
23. Wang, W.; Wu, C.; Bai, T.; Wang, Z.; Yuan, X.; Cui, H. A Pattern Translation Method for Flags in Binary Translation. *Jisuanji Yanjiu Yu Fazhan/Comput. Res. Dev.* **2014**, *51*, 2336–2347. [\[CrossRef\]](#)
24. Wang, R.; Meng, J.; Chen, Z.; Yan, X. Condition Code Optimization in Dynamic Binary Translation. *J. Zhejiang Univ. (Eng. Sci.)* **2014**, *48*, 124–129.
25. Saagarjha TSOEnabler. Available online: <https://github.com/saagarjha/TSOEnabler> (accessed on 16 June 2023).
26. Dougallj Why Is Rosetta 2 Fast. Available online: <https://dougallj.wordpress.com/2022/11/09/why-is-rosetta-2-fast/> (accessed on 26 May 2023).
27. A64–Base Instructions (Alphabetic Order). Available online: <http://hehezhou.cn/isa/> (accessed on 16 June 2023).
28. Asanovic, K. RISC-V “V” Vector Extension. Version 1.0. 2021. Available online: <https://github.com/riscv/riscv-v-spec/releases/tag/v1.0> (accessed on 16 June 2023).
29. XuanTie C908. Available online: <https://xrvn.com/cpu-details?id=4107904466789928960> (accessed on 16 June 2023).
30. Frame, A. Introduction To SiFive Vector Processors. Available online: <https://www.sifive.com/blog/introduction-to-sifive-vector-processors> (accessed on 16 June 2023).
31. Asanovic, K. Cost of Vsetvl Instructions #642. Available online: <https://github.com/riscv/riscv-v-spec/issues/642> (accessed on 16 June 2023).
32. EFLAGS Cross-Reference and Condition Codes. Available online: <https://www.cs.utexas.edu/~byoung/cs429/condition-codes.pdf> (accessed on 16 June 2023).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.